

Apuntes_Pandas_1_Conceptos_basicos

May 11, 2026

1 Apuntes Pandas 1: Conceptos basicos de la libreria Pandas

1.1 ¿Que es Pandas?

Pandas es una libreria creada en Python diseñada para utilizarlo en el trabajo de gestión de datos o big data, **con el objetivo de analizar y manipular datos** de forma rapida, sencilla y flexible.

Es la libreria mas extendida en uso y con mas recursos en Python para el procesamiento y analisis de datos.

Se compone de estructuras de datos y herramientas de manipulacion asi como analisis, potentes, flexibles, sencillas de usar que facilitan muchas tareas cotidianas para un cientifico de datos.

Pandas esta basada en NumPy, pero amplia las funcionalidades.

Ya que dispones de los elementos base de Python, junto a los diferentes tipos de datos que añade y sus funciones, pero Pandas

Algunas de estas funcionalidades que amplia y en las que te ayudara son:

- Leer y cargar datos de diferentes tipos de fuentes. (JSON, CSV, SQL, HTML, XML...)
- Manejar estructuras de datos de tipo longitudinal (series o arrays de datos) y tabular.
- Operaciones de limpieza, filtrado y transformación eficiente de datos.
- Operaciones de agregación y ordenación.
- Operaciones de cruce y fusión de múltiples datos.
- Exportar resultados con distintos tipos de formatos.

Para hacer hacer uso de la libreria, habra que importarla:

```
import pandas as pd
```

Por convencion se pone el alias *pd*

```
[5]: import pandas as pd
```

1.2 Tipos de datos en pandas

Antes de pasar a las estructuras con las que se trabaja en pandas, vamos a mencionar los tipos de datos, el *type* de los datos que van a componer las estructuras de datos.

Estos seran los mismos o estaran basados en los tipos de datos de NumPy

Estos son:

- *object*: Es el equivalente a *str*, una cadena de texto
- *int64* y *float64*: Datos numéricos para representar enteros y decimales
- *bool*: Datos booleanos para comprobar evaluaciones True or False
- *datetime64*, *timedelta64*: Datos formato MM/DD/YYYY para fechas y horas/minutos.
- *category*: Valores *str* limitados a una lista de valores posibles

Pandas reconoce automáticamente el tipo de dato en las estructuras, sin embargo, para los tipos de datos de **fechas o tiempo** y las **category** va a ser necesario **realizar algún cambio de tipo de dato** que ayude al sistema a entenderlo

1.2.1 Conversion de tipos de datos: `.astype()`, `to_numeric()`, `to_datetime()`

Como comentábamos, puede haber algún dato que es necesario que sea entendido de un tipo en concreto, mientras que pandas lo reconoce de otro y nos impide poder trabajar con él adecuadamente.

Ej: Una fecha que queremos que sea tipo *datetime64* esta siendo interpretado como *object* y no nos deja realizar filtrados por las fechas.

Por ello es importante entender las herramientas de **conversion de datos**, existen 2 formas para ello:

- `.astype()`: Cambia la interpretacion del dato al que le indiquemos como parametro
`"nombre serie/dataframe"["nombre columna a cambiar tipo de dato"]`
`= "nombre serie/dataframe"["nombre columna a cambiar tipo de dato"].astype("tipo de dato al que se quiere cambiar")`
- `.to_numeric()` o `.to_datetime()`: Cambia el dato a un tipo numerico o a un tipo de tiempo, de manera mas simple.

Tienen como características comunes:

- Estos metodos sirven para datos que estan **tanto en series como en dataframes**
- La interpretacion de los datos, tal y como funciona pandas, **se realiza basado en columnas** por lo que se convertiran los datos de estas.

```
[18]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8], "fechas": ["2024-05-15", "2024-12-15", "2024-09-20", "2024-03-15", "2025-02-03", "2024-12-28", "2023-07-12", "2024-11-15"]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado.dtypes)
print(40*" -")
cambiado["fechas"]= cambiado["fechas"].astype("datetime64[ns]")
print(cambiado.dtypes)
```

```
Decimas    float64
fechas      object
dtype: object
```

Decimas float64
fechas datetime64[ns]
dtype: object

Cuando usar `.astype()` o funciones `to_..` Hay que hacer una distinción de estos 2 tipos de métodos ya que tienen ciertas limitaciones **cuando la conversión no se puede realizar**

- `.astype()` puede generar una excepción si no se puede realizar una conversión del dato, por lo que **podemos capturarla y gestionarla**
- `.astype()` para convertir ciertos tipos de datos que disponen de su propio método `to_..` están optimizados para la conversión, **por lo que será mejor utilizarlos**

Ej: Si queremos convertir un formato a fecha con `.astype()` nos pedirá que indiquemos el parámetro (`"datetime64[ns]"`) mientras que usando `to_datetime()` no nos pedirá nada.

- `.to_...` cuando no puede convertir nos permite definir que hacer en situaciones donde no se puede convertir el dato, ya que nos permite definir parámetros para gestionar estos *fallos de conversión* (captura la excepción y nos permite trabajar)

`errors = ignore` nos permite ignorar los errores

`coerce = NaN` nos permite gestionar los errores como Nulos y cambiar estos valores

Prefijos para usar con tipos de datos específicos: `.accessors`. Pandas incluye unos tipos de métodos que nos **facilitan el acceso a los datos pero no realizan ninguna modificación** mediante los **`.accessors`**

En general cuando quieras acceder a datos puedes acceder mediante el uso de corchetes, para acceder a columnas, filas o elementos concretos, podemos realizar *mascaras booleanas* para filtrar esa selección de datos... **pero no a todos los tipos de datos se puede acceder así**

Hay tipos de datos que necesitas de un *prefijo* para ayudarle a interpretar el tipo y así poder trabajar con ellos, este prefijo son los *accessors*.

Sin estos `accessors`, no podremos trabajar con algunos tipos de datos

columna/serie.accessor.metodo

Existen 3 `accessors` para estos datos:

- `dt`: permite trabajar con datos de **datetime64**
- `str`: permite trabajar con datos de **object**
- `cat`: permite trabajar con datos de **category**

Trabajar con datos `datetime64` Utilizaremos el `accessor dt` lo que nos permitirá **gestionar fechas y acceder a datos concretos con sus métodos o atributos específicos**, los más interesantes son:

- **`.dt.date()`**: devuelve la fecha completa
- **`.dt.day()`**: devuelve el día de la fecha
- **`.dt.dayofweek()`**: devuelve el día de la semana pero en número (0 = lunes , 6 = domingo)
- **`.dt.month_name()`**: devuelve el nombre del mes

- `.dt.day_name()`: devuelve el nombre del día de la semana

```
[22]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8], "fechas": ["2024-05-15", "2024-12-15", "2024-09-20", "2024-03-15", "2025-02-03", "2024-12-28", "2023-07-12", "2024-11-15"]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado.dtypes)
print(40*"-----")
cambiado["fechas"]= cambiado["fechas"].astype("datetime64[ns]")
print(cambiado["fechas"].dt.day)
```

```
Decimas    float64
fechas      object
dtype: object
-----
Enteros
2      15
3      15
5      20
4      15
6       3
6      28
8      12
8      15
Name: fechas, dtype: int32
```

```
[23]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8], "fechas": ["2024-05-15", "2024-12-15", "2024-09-20", "2024-03-15", "2025-02-03", "2024-12-28", "2023-07-12", "2024-11-15"]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado.dtypes)
print(40*"-----")
cambiado["fechas"]= cambiado["fechas"].astype("datetime64[ns]")
print(cambiado["fechas"].dt.month_name())
```

```
Decimas    float64
fechas      object
dtype: object
-----
Enteros
2      May
3    December
```

```

5     September
4         March
6     February
6     December
8         July
8     November
Name: fechas, dtype: object

```

Trabajar con datos *string* Utilizaremos el accessor *str* lo que nos permitira **transformar la salida de los datos así como comprobar si contiene/comienza/termina con un substring**, los mas interesantes son:

- *.str.lower()*: convierte la cadena de texto a minusculas
- *.str.upper()*: convierte la cadena de texto a mayusculas
- *.str.contains()*: evalua y devuelve un True si contiene un *str* dado como parametro.
- *.str.startswith()*: evalua y devuelve un True si comienza un *str* dado como parametro.

```

[24]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8] , "fechas": ["2024-05-15", "2024-12-15", "2024-09-20", "2024-03-15", "2025-02-03", "2024-12-28", "2023-07-12", "2024-11-15"]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado.dtypes)
print(40*"~")
print(cambiado["fechas"].str.contains("2024"))

```

```

Decimas    float64
fechas      object
dtype: object
-----
Enteros
2      True
3      True
5      True
4      True
6     False
6      True
8     False
8      True
Name: fechas, dtype: bool

```

Trabajar con datos *category* Utilizaremos el accessor *cat* lo que nos permitira **trabajar con datos category**, los mas interesantes son:

- *.cat.categories*: devuelve la lista de categorias disponibles.
- *.cat.codes*: transforma las categorias en codigos numericos.
- *.cat.ordered*: evalua y devuelve un True si la columna category esta ordenada.

```
[29]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8], "fechas": ["2024-05-15", "2024-12-15", "2024-09-20", "2024-03-15", "2025-02-03", "2024-12-28", "2023-07-12", "2024-11-15"], "Nombres": ["miguel", "pablo", "juan", "carlos", "paloma", "nataly", "pepe", "paco"]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado.dtypes)
print(40*" -")
cambiado["Nombres"]= cambiado["Nombres"].astype("category")
print(cambiado["Nombres"].cat.codes)
```

```
Decimas    float64
fechas      object
Nombres     object
dtype: object
```

```
-----
Enteros
```

```
2    2
3    4
5    1
4    0
6    6
6    3
8    7
8    5
```

```
dtype: int8
```

1.3 Tipos de estructuras de datos en pandas

Existen elementalmente 2 estructuras de datos (objetos) en pandas:

- las *Series* para datos unidimensionales, es decir, vectores o arrays de 1 dimension
- los *Dataframes* para datos tabulares, es decir, matrices o arrays de 2 dimensiones

En la libreria son **clases propias** por los que podemos llamarlos directamente si los importamos para crear los objetos

```
from pandas import Series, DataFrame
```

1.3.1 *Series*

Se puede definir como un tipo de estructura de dato (objeto) de 1 dimension, que nace de los Array, y **tienen una etiqueta que lo definen**

Los objetos Series almacenan tanto la secuencia de valores como una secuencia de etiquetas asociadas a cada elemento

Las series estan compuestas por 2 elementos basicos:

- *data*: serán los datos (arrays) con los que operaremos
- *index*: serán los índices de los arrays, **la etiqueta que pondremos a los datos** > En Numpy los índices de los arrays siempre son numéricos, nos referimos a ellos como [0] a [n-1], **en pandas no pueden ser de diferentes tipos**.

Se expresara así:

```
"nombre serie" = pd.Series (data = "nombre variable con datos" , index = ["etiquetas para los datos"])
```

Tienen estas características:

- Como en Numpy, solo admiten **un tipo de valor por serie** > Es decir si incluyes varios elementos de diferentes tipos de datos, te escogera el que mejor cumpla la condición de tipo de dato con todos los elementos > > > Esto es debido a que las series están basados en los Numpy Arrays y tienen esta característica
- El índice por defecto es un range (0,N)
- El índice debe tener **el mismo tamaño que los datos** > Es decir, no puede haber más elementos en el índice que datos vayamos a etiquetar
- El índice puede ser de tipo *Integer* o *String*
- Se pueden crear Series desde: > **Listas []** > > **Arrays ([])** > > **Diccionarios {}** > > > En los diccionarios por defecto **los índices son las claves**

```
[6]: import pandas as pd

datos = [45,32,42]
prueba = pd.Series (data=datos, index = ["Numero 1", "Numero 2", "Numero 3"])
print(prueba)
```

```
Numero 1    45
Numero 2    32
Numero 3    42
dtype: int64
```

```
[10]: from pandas import Series, DataFrame

datos = [45,32,42]
prueba = Series(data=datos, index = ["Numero 1", "Numero 2", "Numero 3"])
print(prueba)
```

```
Numero 1    45
Numero 2    32
Numero 3    42
dtype: int64
```

Si generamos una serie nueva que no disponga de **nombre para el dataset** podremos denominarlo así:

```
"nombre serie" = pd.Series ("funcion para generar array de datos", name = "nombre del data set")
```

1º Consideracion: **tienes que importar la libreria *numpy* as *np*** para poder utilizarlo

2º Consideracion: **el nombre de la variable es distinto al del dataset**

```
[7]: import numpy as np
from pandas import Series, DataFrame

prueba = Series(np.linspace(3,10,9), name="casa")
print(prueba)
```

```
0    3.000
1    3.875
2    4.750
3    5.625
4    6.500
5    7.375
6    8.250
7    9.125
8   10.000
```

Name: casa, dtype: float64

1.3.2 Dataframes

Se puede definir como un tipo de estructura de datos **de 2 dimensiones** (fila/columna) que nace de los Array, que sirve para representar datos tabulares (*en tabla*).

Las series estan compuestas por 2 elementos basicos:

- *data*: seran los datos (arrays) con los que operaremos
- *index*: seran los indices de las filas
- *columns*: seran los indices de las columnas

Se debe utilizar esta etiqueta para **indicar que tipo de dato se vera en esa columna**

Cuando utilizamos diccionarios, **las claves seran por defecto las *columns* de los datos**

Si no defines las columns, **se pondran en orden alfabetico por defecto**

Se expresara asi:

```
"nombre serie" = pd.DataFrame (data = "nombre variable con datos" ,
index = ["etiquetas para las filas"], columns = ["etiquetas para las columnas"])
```

Tienen estas caracteristicas:

- Poseen 2 **ejes etiquetados** (filas y columnas)
- Las columnas pueden ser imaginadas como una Serie
- Se pueden crear DataFrames desde: > **CVS** > > **JSON** > > **Diccionarios {}** > > **Tuplas ()** > > **Arrays ([])**


```
[9]: import pandas as pd

datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0]}
prueba = pd.DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3"])
print(prueba)
```

	Enteros	Decimas
Numero 1	2	0.4
Numero 2	3	0.6
Numero 3	5	0.0

```
[12]: from pandas import Series, DataFrame

datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0]}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3"])
print(prueba)
```

	Enteros	Decimas
Numero 1	2	0.4
Numero 2	3	0.6
Numero 3	5	0.0

- Puedes definir una DataFrame con un diccionario **que incluya en los valores Arrays**

“nombre diccionario para dataframe” = {“clave”:“nombre array”}

Muy util a la hora de combinar datos de diferentes origenes

```
[62]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
print(prueba)
```

	Enteros	Decimas	milesimas
Numero 1	2	0.4	4
Numero 2	3	0.6	7
Numero 3	5	0.0	2

- Se pueden saber **los indices y columnas asociados a un DataFrame**

“nombre dataframe”.index

Vale tambien para para las series

“nombre dataframe”.columns

```
[63]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
```

```
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.index
```

[63]: Index(['Numero 1', 'Numero 2', 'Numero 3'], dtype='object')

```
[64]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.columns
```

[64]: Index(['Enteros', 'Decimas', 'milesimas'], dtype='object')

- Se pueden saber la forma, el tamaño y la cuenta de elementos no nulos de un DataFrame

“nombre dataframe”.shape

Para sacar las filas es mas facil utilizar **un posicionador [0]**

“nombre dataframe”.size

“nombre dataframe”.count()

```
[65]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.shape
```

[65]: (3, 3)

```
[66]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.size
```

[66]: 9

```
[70]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.count()
```

```
[70]: Enteros      3
      Decimas      3
      milesimas    3
      dtype: int64
```

- Se pueden saber las primeras y/o las ultimas filas de un DataFrame

“nombre dataframe”.head

“nombre dataframe”.tail

Coge por defecto 5 filas

```
[69]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.head()
```

```
[69]:      Enteros  Decimas  milesimas
Numero 1      2      0.4          4
Numero 2      3      0.6          7
Numero 3      5      0.0          2
```

- Se pueden añadir columnas a un Dataframe ya creado, hay 2 posibilidades:

Que todas las filas tengan el mismo valor

“nombre dataframe”[“nombre columna nueva”] = “valor”

Que las filas tengan distintos valores

“nombre dataframe”[“nombre columna nueva”] = “operadores matematicos o valores”
“nombre dataframe”[“nombre columna existente”]

```
[7]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba["Centesimas"] = 0.5
print(prueba)
```

```
      Enteros  Decimas  milesimas  Centesimas
Numero 1      2      0.4          4          0.5
Numero 2      3      0.6          7          0.5
Numero 3      5      0.0          2          0.5
```

```
[8]: from pandas import Series, DataFrame
```

```

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba["Centesimas"] = 0.5
prueba["Multiplo infi."] = prueba["Centesimas"] * prueba["Decimas"] *
↪prueba["milesimas"] * prueba["Enteros"]
print(prueba)

```

	Enteros	Decimas	milesimas	Centesimas	Multiplo infi.
Numero 1	2	0.4	4	0.5	1.6
Numero 2	3	0.6	7	0.5	6.3
Numero 3	5	0.0	2	0.5	0.0

1.4 Indexacion, Slicing y mas funciones para trabajar datos.

Una vez vistas las estructuras de datos de pandas, toca entender las posibilidades de interaccion con esos datos, los comportamientos de los mismos y las funciones utilizables para en definitiva poder extraer informacion para interpretar esos datos y sacar conclusiones de nuestro analisis.

Primero, los comportamientos de estas estructuras son:

- *Indexacion* o acceso a los datos
- *Slicing* o captura o seleccion de datos

Estos 2 comportamientos de las estructuras de datos **son las basicas** ya que en la mayoria de los casos, del origen de la captacion de datos, deberemos extraer algunas series o dataframes de datos y no sera necesario **obtener todos los datos disponibles** por lo que acceder y cortar datos sera los conceptos basicos de interaccion.

Ahora como se utilizan estos comportamientos y otras funciones que se pueden utilizar con las estructuras de datos de pandas.

Acceso a datos mediante rebanadas Al igual que sucedia en NumPy podemos hacer una seleccion de datos de una manera sencilla con las rebanadas pero *adaptandose a pandas*

- Acceder a datos de **series por su posicion**

“nombre serie”[posicion numerica]

Funciona si no hemos definido **los indices al crear la serie** ya que se ponen por defecto los numeros

```

[51]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = ["padre", "nieto", "abuelo"]
my_serie = Series(lista)
print(my_serie)
print(40*"-")
print( my_serie[0])

```

```
0    padre
1    nieto
2    abuelo
dtype: object
```

padre

- Acceder a datos de **series o dataframes**, para **indices**, mediante **rebanadas por posicionadores o por etiquetas**

“nombre serie/dataframe”[start numerico :stop numerico]

Funciona si no hemos definido **los indices al crear la serie** ya que se ponen por defecto los numeros

Recuerda que coge **n-1** de las posiciones que le indicas

“nombre serie/dataframe”[start “etiqueta” :stop “etiqueta”]

```
[74]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = ["padre", "nieto", "abuelo"]
my_serie = Series(lista, index= ["pedro", "luis", "carlos"])
print(my_serie)
print(40*"-")
print( my_serie["padre":"abuelo"])
```

```
pedro    padre
luis     nieto
carlos   abuelo
dtype: object
```

```
luis     nieto
carlos   abuelo
dtype: object
```

```
[ ]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = ["padre", "nieto", "abuelo"]
my_serie = Series(lista)
print(my_serie)
print(40*"-")
print( my_serie[:2])
```

- Acceder a datos de **series por su indice** (un valor o una lista de valores)

“nombre serie”[“nombre del indice asociado al valor o una lista de valores”]

```
[80]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = ["padre", "nieto", "abuelo"]
my_serie = Series(lista, index= ["pedro", "luis", "carlos"])
print(my_serie)
print(40*"-")
print( my_serie[0])
```

```
pedro      padre
luis       nieto
carlos     abuelo
dtype: object
```

```
-----
padre
```

C:\Users\insau\AppData\Local\Temp\ipykernel_21036\824318426.py:9: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
print( my_serie[0])
```

- Acceder a datos de **series o dataframes mediante mascaras booleanas**

“nombre serie”[“nombre serie” “</>/=/...” “valor comparativo”]

“nombre dataframe”[“nombre dataframe.”etiqueta columna” “</>/=/...” “valor comparativo”]

Si usas un nombre (o una lista de nombres) **entre corchetes []**, seleccionas columnas.

Si indicas un rango o rebanada usando **‘:’**, o una máscara booleana, seleccionas filas.

```
[56]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = [12, -1, None]
my_serie = Series(lista, index= ["Primero", "Segundo", "Tercero"])
print(my_serie)
print(40*"-")
print( my_serie[my_serie > 0])
```

```
Primero    12.0
Segundo    -1.0
Tercero     NaN
dtype: float64
-----
```

```
Primero    12.0
dtype: float64
```

```
[77]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
print( prueba[prueba.milesimas > 5 ])
```

```
      Enteros  Decimas  milesimas
Numero 2      3      0.6          7
```

- Acceder a datos de Dataframes por su columna mediante posicionadores o por atributo (un valor o una lista de valores)

“nombre dataframe”[“nombre de la columna asociado al valor o una lista de valores”]

Esta opcion permite ademas elegir una lista de nombres de columnas a la vez usando dobles corchetes

“nombre dataframe”[[“lista con nombres de columnas”]]

“nombre dataframe”.“nombre de la columna asociado al valor o una lista de valores”

No es recomendable ya que el nombre de la columnas puede confundirse con otra variable, metodo, clase,...

```
[72]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba["Enteros"]
```

```
[72]: Numero 1    2
      Numero 2    3
      Numero 3    5
      Name: Enteros, dtype: int64
```

```
[73]: from pandas import Series, DataFrame

array= ([4,7,2])
datos = {"Enteros": [2, 3, 5], "Decimas": [.4, .6, .0], "milesimas":array}
prueba = DataFrame (data=datos, index = ["Numero 1", "Numero 2", "Numero 3",])
prueba.Enteros
```

```
[73]: Numero 1    2
      Numero 2    3
```

Numero 3 5
Name: Enteros, dtype: int64

Acceso a datos mediante *loc()* y *iloc* Para acceder a los datos en pandas, se utilizan principalmente 2 metodos:

- *loc()*: Nos permitira indexar o “localizar” los datos por **columnas** y utilizaremos las **etiquetas** para localizarlas > *“nombre dataframe”.loc([“nombre indice”, “nombre etiqueta o columna”])*
- *iloc()*: Nos permitira indexar o “localizar” los datos por **filas** y utilizaremos el **indice numerico (como en los array)** para localizarlas. > *“nombre dataframe”.iloc([posicion numerica indice, posicion numerica columna])*

Tiene algunas características que podremos utilizar a nuestro favor:

- Se pueden utilizar *las mascaras booleanas* para seleccionar datos, lo que posibilita **un filtrado sencillo y rapido** > Para obtener el resultado **filtrado final de los datos** > > > *“nombre serie/dataframe”.iloc/.loc[“nombre serie/dataframe”[“etiqueta que comparara”] </>/=/!=... “valor comparativo”]* > > Para obtener el resultado **booleano (True o False)** de los datos > > > *“nombre serie/dataframe”.iloc/.loc[“etiqueta que comparara”] </>/=/!=... “valor comparativo”]*
- Para localizar **una fila en concreto** se utiliza esta forma: > *“nombre dataframe”.loc([“nombre indice”,])* > > *“nombre dataframe”.iloc([posicion numerica start indice,])*
- Para localizar **una columna en concreto** se utiliza esta forma: > *“nombre dataframe”.loc([, “nombre etiqueta o columna”])* > > > *“nombre dataframe”.iloc[:, posicion numerica columna])*
- Para seleccionar **un subconjunto de datos** se utiliza esta forma: > *“nombre dataframe”.loc[“nombre indice start” : “nombre indice stop”, [“nombre etiqueta o columna1”, “nombre etiqueta o columna2”]]* > > *“nombre dataframe”.iloc[“posicion numerica indice start” : “posicion numerica indice stop”, “posicion numerica columna start” : “posicion numerica columna stop”]]*

```
[42]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

diccionario = {"Nombres ": ["Pedro", "Juan", "Carlos"] , "Edades": [30,12,53] ,
↳ "Sexo": ["Señor","Señor","Señor"]}
lista = ["padre", "nieto", "abuelo"]
my_dataframe = DataFrame (diccionario)
my_serie = Series(lista, name = "rango familiar")
my_contact = pd.concat([my_dataframe,my_serie], axis = 1)
print(my_contact)
indexado = my_contact.set_index("rango familiar")
print(indexado)
```



```
print(40*"~")
print( indexado.loc["padre"])
```

	Nombres	Edades	Sexo	rango familiar
0	Pedro	30	Señor	padre
1	Juan	12	Señor	nieta
2	Carlos	53	Señor	abuelo

	Nombres	Edades	Sexo
rango familiar			
padre	Pedro	30	Señor
nieta	Juan	12	Señor
abuelo	Carlos	53	Señor

Nombres Pedro
 Edades 30
 Sexo Señor
 Name: padre, dtype: object

```
[47]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

diccionario = {"Nombres ": ["Pedro", "Juan", "Carlos"] , "Edades": [30,12,53] ,
↳ "Sexo": ["Señor","Señor","Señor"]}
lista = ["padre", "nieta","abuelo"]
my_dataframe = DataFrame (diccionario)
my_serie = Series(lista, name = "rango familiar")
my_contact = pd.concat([my_dataframe,my_serie], axis = 1)
print(my_contact)
indexado = my_contact.set_index("rango familiar")
print(indexado)
print(40*"~")
print( indexado.iloc[0])
```

	Nombres	Edades	Sexo	rango familiar
0	Pedro	30	Señor	padre
1	Juan	12	Señor	nieta
2	Carlos	53	Señor	abuelo

	Nombres	Edades	Sexo
rango familiar			
padre	Pedro	30	Señor
nieta	Juan	12	Señor
abuelo	Carlos	53	Señor

Nombres Pedro
 Edades 30
 Sexo Señor
 Name: padre, dtype: object

- Podemos utilizar estas funciones de seleccion **para operar matematicamente con elementos del DataFrame** mediante operadores matematicos simples (+,-,*...)

“nombre serie/dataframe”.loc[“nombre o rebanada de indices”, “nombre de columnas”] = “nombre serie/dataframe”.iloc/.loc[“nombre o rebanada de indices”, “nombre de columnas”] “operador matematico” “valor”

```
[5]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

diccionario = {"Nombres ": ["Pedro", "Juan", "Carlos"] , "Edades": [30,12,53] ,
↳"Sexo": ["Señor","Señor","Señor"]}
lista = ["padre", "nieto", "abuelo"]
my_dataframe = DataFrame (diccionario)
my_serie = Series(lista, name = "rango familiar")
my_contact = pd.concat([my_dataframe,my_serie], axis = 1)
print(my_contact)
indexado = my_contact.set_index("rango familiar")
print(indexado)
print(40*" -")
indexado.loc["padre":"abuelo", "Edades"] = indexado.loc["padre":"abuelo",
↳"Edades"] -10
print(indexado)
```

	Nombres	Edades	Sexo	rango familiar
0	Pedro	30	Señor	padre
1	Juan	12	Señor	nieto
2	Carlos	53	Señor	abuelo

	Nombres	Edades	Sexo
rango familiar			
padre	Pedro	30	Señor
nieto	Juan	12	Señor
abuelo	Carlos	53	Señor

	Nombres	Edades	Sexo
rango familiar			
padre	Pedro	20	Señor
nieto	Juan	2	Señor
abuelo	Carlos	43	Señor

1.4.1 Funciones para utilizar en pandas

A continuación indicaremos algunas funciones utilizadas de la libreria de pandas que nos ayudaran en nuestras tareas.

Pandas introduce varios grupos de funciones propias que vamos a ver: *Conversion CSV* , *Gestion de indices*, *Deteccion de Nulos (NaN)*, *Agrupacion*, *Operaciones*

Conversion CSV Modificaran los datos para utilizarlos o devolverlos a archivos tipos CSV.

2 funciones a conocer son:

- `read_csv()`: Nos permite crear DataFrames desde un CSV
- `to_csv()`: Nos permite crear un CSV desde un DataFrame

Estas 2 funciones nos permitirán en definitiva **poder mostrar conclusiones de nuestro análisis de datos**

Gestión de índices Modifica los índices de los datos con los que trabajamos.

2 funciones a conocer son:

- `.reset_index()`: Cambia el índice actual y utiliza el generado por defecto (*numérico 0 a n-1*)
- `.set_index()`: Permite definir una columna como nuevo índice.

En esta función, nos permite establecer **múltiples índices** y nos los ordenará con los datos que tenemos.

`.set_index(["nombre índice 1", "nombre índice 2",...])`

```
[10]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0], "Enteros": [2, 3, 5] , "Milesimas": [0.03, 0.06, 0.08]}
prueba = DataFrame (data=datos)
print(prueba)
print(40*"---")
cambiado= prueba.set_index("Enteros")
print(cambiado)
```

	Decimas	Enteros	Milesimas
0	0.4	2	0.03
1	0.6	3	0.06
2	0.0	5	0.08

	Decimas	Milesimas
Enteros		
2	0.4	0.03
3	0.6	0.06
5	0.0	0.08

Detección de Nulos Permite gestionar series y dataframes que no tengan datos.

- `isnull()`: Devuelve *True* para valores donde no hay valores (NaN)
- `notnull()`: Devuelve *False* para valores donde no hay valores (NaN)
- `dropna()`: Elimina todos los valores donde no hay valores (NaN)

Por defecto elimina las filas donde hay datos nulos

Es posible configurarlo para eliminar **columnas** así:

dropna(axis="columns")

- *fillna()*: Da un valor definido como parametro a los valores donde no hay valores (NaN)

Agrupacion

- *groupby*

Nos permitira agrupar los datos por uno o mas campos

Se expresa asi:

"nombre dataframe".groupby("etiqueta columna")."funcion operacion"()

Caracteristicas:

- Nos facilita la gestión de gran cantidad de datos
- Nos permite ejecutar cualquier operación sobre esos datos (suma, media, número de ocurrencias...)
- *concat()*

Nos permite concatenar 2 series o dataframes para unir los datos

pd.concat(["nombre serie1", "nombre serie2",...])

En estos casos es recomendable utilizar un funcion **de gestion de indice** ya que los indices de los datos tienen un aspecto desordenado u heterogeneo.

```
[20]: from pandas import Series, DataFrame

datos = {"Decimas": [.4, .6, .0, .7, .8, .9, .5, .8], "Enteros": [2, 3, 5, 4, 6, 6, 8, 8], "Milesimas": [0.03, 0.06, 0.08, .07, .05, .03, .08, .02]}
prueba = DataFrame (data=datos)
cambiado= prueba.set_index("Enteros")
print(cambiado)
print(40*" -")
agrupacion = cambiado.drop("Milesimas", axis = "columns")
agrupacion2= agrupacion.groupby("Enteros").describe()
print(agrupacion2)
```

	Decimas	Milesimas
Enteros		
2	0.4	0.03
3	0.6	0.06
5	0.0	0.08
4	0.7	0.07
6	0.8	0.05
6	0.9	0.03
8	0.5	0.08
8	0.8	0.02

	Decimas							
	count	mean	std	min	25%	50%	75%	max
Enteros								
2	1.0	0.40	NaN	0.4	0.400	0.40	0.400	0.4
3	1.0	0.60	NaN	0.6	0.600	0.60	0.600	0.6
4	1.0	0.70	NaN	0.7	0.700	0.70	0.700	0.7
5	1.0	0.00	NaN	0.0	0.000	0.00	0.000	0.0
6	2.0	0.85	0.070711	0.8	0.825	0.85	0.875	0.9
8	2.0	0.65	0.212132	0.5	0.575	0.65	0.725	0.8

Operaciones Se incluyen las operaciones que ya existian en NumPy y las amplia , ya sean matematicas o de estadistica.

- Las matematicas incluyen funciones nuevas como *add*, *sub*, *mul*
- Las estadisticas incluyen funciones como *mean*, *sum*, *std*, *count*, *median*,...

La mas interesante a conocer es la funcion **.describe()** que nos indica distintos parametros (imagen con ejemplo)

```

          altura
count    6.000000
mean     1.753333
std      0.081650
min      1.670000
25%      1.692500
50%      1.730000
75%      1.812500
max      1.870000

```

- Se pueden realizar **operaciones matematicas con series y un scalar** (sumar, restar, multiplicar, dividir, resto,...)

```
[57]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = [12, -1, 5.2]
my_serie = Series(lista, index= ["Primero", "Segundo", "Tercero"])
print(my_serie)
print(40*"-")
print( my_serie*2)
```

```
Primero    12.0
Segundo    -1.0
Tercero     5.2
dtype: float64
```

```
-----
Primero    24.0
Segundo    -2.0
Tercero    10.4
dtype: float64
```

- Se pueden realizar **operaciones matematicas con series elemento a elemento** (sumar, restar, multiplicar, dividir, resto,...)

```
[59]: import numpy as np
import pandas as pd
from pandas import Series, DataFrame

lista = [12, -1, 5.2]
lista2 = [5, 6, -4.3]
my_serie = Series(lista, index= ["Primero", "Segundo", "Tercero"])
my_serie2 = Series(lista2, index= ["Primero", "Segundo", "Tercero"])
print(my_serie)
print(40*"-")
print( my_serie + my_serie2)
```

```
Primero    12.0
Segundo    -1.0
Tercero     5.2
dtype: float64
```

```
-----
Primero    17.0
Segundo     5.0
Tercero     0.9
dtype: float64
```

- Se pueden realizar **otras operaciones matematicas con series**

```
[60]: import numpy as np
import pandas as pd
```

```

from pandas import Series, DataFrame

lista = [12, -1, 5.2]
my_serie = Series(lista, index= ["Primero", "Segundo", "Tercero"])
print(my_serie)
print(40*"-")
print( my_serie.prod())

```

```

Primero      12.0
Segundo      -1.0
Tercero       5.2
dtype: float64
-----
-62.400000000000006

```

Otros Existen otras funciones que sirven para la gestion de los datos:

- **.drop():** Oculta o salta un eje de un dataset para no utilizarlo
“nombre dataframe”.drop(“etiqueta a ocultar”, axis=“columns”/“index”)